	NAME	MOHAMMAD IRSYAD HAKIMI BIN SABARUDIN	
	REGISTRATION NO	25DIT24F1089	**MARKS** / 100
	SESSION	SESI 1 2026/2027	
	CLASS	DIT5F	
	CODE/SUBJECT	DFK50083 / Python Programming	
	PRACTICAL WORK	4	

CLO 1: Display suitable techniques in mathematical computation and software application using python programming. (P3, PLO3)

TOPIC: 4.1 Explain the Introduction to Matrix Processing

4.2 Explain Analysis and manipulation tools DURATION:

3 HOURS

INSTRUCTION:

Construct a Python program using NumPy and Pandas with appropriate comments. Display all outputs clearly.

Case study: PMJ PolyCafe Sales Analysis System

PMJ PolyCafe serves food and drinks to students of Politeknik Mersing. The cafe management needs a simple Python program to record student purchases and analyse sales. Registered PMJ PolyCafe members receive a 10% discount on their purchases, while non-members pay the normal price. You are required to use NumPy and Pandas to process, organize, manipulate and analyse the data.

Use the following menu in your program:

Code	Item	Category	Price (RM)
1	Nasi Lemak	Food	4.00
2	Chicken Rice	Food	6.50
3	Mee Goreng	Food	5.00
4	Iced Milo	Drink	3.00
5	Mineral Water	Drink	1.50

1. Matrix Processing using NumPy (CLO1, P3) - 30 marks

Construct a Python program using NumPy to perform the following tasks: i.
Ask the user to enter:

- Student Name
- Menu Item Code
- Number of items purchased (integer)

- Membership Status (Yes or No) ii. Determine the Item Name, Category and Price per Item based on the PMJ PolyCafe menu and store the values in suitable variables.

iii. Create a NumPy array using np.full() with the following requirements:

- Array size must be based on the number of items purchased.
- Each element must contain the price of the selected item. iv. Display the following:
- The generated NumPy array
- The first element of the array using array[0]
- Total Purchase using .sum()

v. Display the following properties of the array:

- Number of dimensions using .ndim
- Shape of the array using .shape vi. Display all results clearly with appropriate messages.

2. Data Organization using Pandas (CLO1, P3) - 20 marks

Using the data obtained from Question 1, perform the following tasks:

i. Create a Pandas DataFrame with the following columns:
   - Student Name
   - Item
   - Category
   - Quantity
   - Membership
   - Price per Item
   - Total Purchase

ii. The first record must use the data entered by the user in Question 1.

iii. Add at least THREE (3) additional records with different student names, items, quantities and membership status. Calculate the Total Purchase for each record.

iv. Display the complete DataFrame clearly.

3. Membership Discount and Final Payment (CLO1, P3) - 15 marks

i. Add a new column named Discount using the following condition:
   - If Membership = Yes, Discount = 10% of Total Purchase.
   - If Membership = No, Discount = RM0.00.

ii. Add a new column named Final Payment using the formula:
   - Final Payment = Total Purchase - Discount iii. Display the updated DataFrame clearly with appropriate messages.

4. Data Manipulation (CLO1, P3) - 15 marks

i. Add ONE duplicate record into the DataFrame by repeating an existing row.

ii. Detect duplicate rows using .duplicated(). iii. Remove duplicate rows using .drop_duplicates(). iv. Insert ONE missing value (None) in the Membership column.

v. Detect missing values using .isnull().

vi. Replace the missing Membership value with "No" using .fillna(). vii. Display the results clearly after each step.

5. Data Analysis (CLO1, P3) - 10 marks

i. Group the cleaned DataFrame by Category using .groupby().

ii. Calculate the following for each category:

- Total Purchase using .sum()
- Total Final Payment using .sum()
- Number of transactions using .count() iii. Display the analysis results clearly.

6. Submission

Submit the following files through the CIDOS platform:

i. Python file (.py): Your_registration_number_PW4.py

ii. PDF file containing Python code, program output/screenshots and rubric: Your_registration_number_PW4.pdf

SOURCE CODE

```
import numpy as np
import pandas as pd

def main():
    print("=" * 65)
    print("         PMJ POLYCAFE SALES ANALYSIS SYSTEM     ")
    print("=" * 65)

    # ---------------------------------------------------------------------
    # PMJ POLYCAFE MENU DATASET
    # ---------------------------------------------------------------------
    menu = {
        1: {"Item": "Nasi Lemak", "Category": "Food", "Price": 4.00},
        2: {"Item": "Chicken Rice", "Category": "Food", "Price": 6.50},
        3: {"Item": "Mee Goreng", "Category": "Food", "Price": 5.00},
        4: {"Item": "Iced Milo", "Category": "Drink", "Price": 3.00},
        5: {"Item": "Mineral Water", "Category": "Drink", "Price": 1.50},
    }

    # Display PMJ PolyCafe Menu
    print("\n--- PMJ POLYCAFE MENU ---")
    print(
        f"{'Code':<6} | {'Item Name':<15} | {'Category':<10} | {'Price (RM)':<10}"
    )
    print("-" * 52)
    for code, details in menu.items():
        print(
            f"{code:<6} | {details['Item']:<15} | {details['Category']:<10} | RM {details['Price']:.2f}"
        )
    print("-" * 52)

    # TASK 1: Matrix Processing using NumPy
    print("\n" + "=" * 65)
    print("TASK 1: MATRIX PROCESSING USING NUMPY")
```

```
print("=" * 65)

# i. Prompt user input
student_name = input("Enter Student Name: ").strip()

# Input validation for Menu Item Code
while True:
    try:
        item_code = int(input("Enter Menu Item Code (1-5): "))
        if item_code in menu:
            break
        else:
            print("Invalid Code! Please enter a number between 1 and 5.")
    except ValueError:
        print("Invalid Input! Please enter a valid integer.")

# Input validation for Quantity
while True:
    try:
        quantity = int(input("Enter Number of items purchased: "))
        if quantity > 0:
            break
        else:
            print("Quantity must be greater than 0.")
    except ValueError:
        print("Invalid Input! Please enter an integer quantity.")

# Input validation for Membership Status
while True:
    membership = input("Membership Status (Yes/No): ").strip().capitalize()
    if membership in ["Yes", "No"]:
        break
    else:
        print("Invalid Status! Please enter 'Yes' or 'No'.")

# ii. Determine Item Name, Category, and Price per Item
```

```
selected_item = menu[item_code]
item_name = selected_item["Item"]
category = selected_item["Category"]
price_per_item = selected_item["Price"]

# iii. Create NumPy array using np.full()
# Array size is based on quantity, filled with price_per_item
price_array = np.full(quantity, price_per_item)

# iv. Display Array, First element array[0], and Total Purchase using .sum()
first_element = price_array[0]
total_purchase_q1 = price_array.sum()

# v. Display Array properties: .ndim and .shape
num_dimensions = price_array.ndim
array_shape = price_array.shape

# vi. Display output clearly with appropriate messages
print("\n--- Task 1 Output ---")
print(f"Student Name            : {student_name}")
print(f"Selected Item           : {item_name} ({category})")
print(f"Price per Item          : RM {price_per_item:.2f}")
print(f"Quantity Purchased      : {quantity}")
print(f"Generated NumPy Array   : {price_array}")
print(f"First Element (array[0]): RM {first_element:.2f}")
print(f"Total Purchase (.sum()) : RM {total_purchase_q1:.2f}")
print(f"Array Dimensions (.ndim): {num_dimensions}")
print(f"Array Shape (.shape)    : {array_shape}")

# TASK 2: Data Organization using Pandas (20 Marks)
print("\n" + "=" * 65)
print("TASK 2: DATA ORGANIZATION USING PANDAS")
print("=" * 65)

# i, ii, iii. Create initial dataset with User input (1st record) + 3 additional records
dataset = [
```

```
{
    "Student Name": student_name,
    "Item": item_name,
    "Category": category,
    "Quantity": quantity,
    "Membership": membership,
    "Price per Item": price_per_item,
    "Total Purchase": total_purchase_q1,
},
{
    "Student Name": "Irsyad ",
    "Item": "Nasi Lemak",
    "Category": "Food",
    "Quantity": 2,
    "Membership": "No",
    "Price per Item": 4.00,
    "Total Purchase": 2 * 4.00,
},
{
    "Student Name": "Ambok irfan",
    "Item": "Iced Milo",
    "Category": "Drink",
    "Quantity": 4,
    "Membership": "Yes",
    "Price per Item": 3.00,
    "Total Purchase": 4 * 3.00,
},
{
    "Student Name": "Fakhrul iman",
    "Item": "Mineral Water",
    "Category": "Drink",
    "Quantity": 5,
    "Membership": "No",
    "Price per Item": 1.50,
    "Total Purchase": 5 * 1.50,
},
```

```
]

df = pd.DataFrame(dataset)

# iv. Display complete DataFrame clearly
print("\n--- Complete Initial DataFrame ---")
print(df.to_string(index=True))

# TASK 3: Membership Discount and Final Payment
print("\n" + "=" * 65)
print("TASK 3: MEMBERSHIP DISCOUNT AND FINAL PAYMENT")
print("=" * 65)

# i. Add 'Discount' column: 10% of Total Purchase if Membership = Yes, else RM0.00
df["Discount"] = df.apply(
    lambda row: (
        row["Total Purchase"] * 0.10
        if row["Membership"] == "Yes"
        else 0.00
    ),
    axis=1,
)

# ii. Add 'Final Payment' column: Final Payment = Total Purchase - Discount
df["Final Payment"] = df["Total Purchase"] - df["Discount"]

# iii. Display updated DataFrame clearly
print(
    "\n--- Updated DataFrame with Discount and Final Payment ---"
)
print(df.to_string(index=True))

# TASK 4: Data Manipulation
print("\n" + "=" * 65)
print("TASK 4: DATA MANIPULATION")
print("=" * 65)
```

```
# i. Add ONE duplicate record into DataFrame by repeating an existing row
duplicate_row = df.iloc[[1]]  # Select row 1 (Irsyad )
df = pd.concat([df, duplicate_row], ignore_index=True)
print("\nDataFrame after adding 1 duplicate row:")
print(df.to_string(index=True))

# ii. Detect duplicate rows using .duplicated()
print("\nDetect Duplicate Rows using .duplicated():")
print(df.duplicated())

# iii. Remove duplicate rows using .drop_duplicates()
df = df.drop_duplicates().reset_index(drop=True)
print(
    "\nDataFrame after removing duplicate rows using .drop_duplicates():"
)
print(df.to_string(index=True))

# iv. Insert ONE missing value (None) in Membership column
df.loc[2, "Membership"] = None
print(
    "\nDataFrame after inserting ONE missing value (None) at Index 2:"
)
print(df.to_string(index=True))

# v. Detect missing values using .isnull()
print("\nDetect Missing Values using .isnull():")
print(df.isnull())

# vi. Replace missing Membership value with "No" using .fillna()
df["Membership"] = df["Membership"].fillna("No")
print(
    "\nDataFrame after replacing missing Membership with 'No' using .fillna():"
)
print(df.to_string(index=True))
```

```
# Recalculate Discount and Final Payment to ensure data consistency
df["Discount"] = df.apply(
    lambda row: (
        row["Total Purchase"] * 0.10
        if row["Membership"] == "Yes"
        else 0.00
    ),
    axis=1,
)
df["Final Payment"] = df["Total Purchase"] - df["Discount"]

# vii. Display final clean DataFrame
print("\nFinal Cleaned DataFrame ---")
print(df.to_string(index=True))

# TASK 5: Data Analysis (10 Marks)
print("\n" + "=" * 65)
print("TASK 5: DATA ANALYSIS (GROUPING)")
print("=" * 65)

# i & ii. Group cleaned DataFrame by Category using groupby()
# Calculate Total Purchase (.sum()), Total Final Payment (.sum()), and Transactions (.count())
analysis_df = (
    df.groupby("Category")
    .agg(
        Total_Purchase=("Total Purchase", "sum"),
        Total_Final_Payment=("Final Payment", "sum"),
        Transaction_Count=("Student Name", "count"),
    )
    .reset_index()
)

# iii. Display analysis results clearly
print("\n--- Sales Analysis by Category ---")
print(analysis_df.to_string(index=False))
```

```
    print("\n" + "=" * 65)
    print("       SYSTEM PROCESSING COMPLETED SUCCESSFULLY       ")
    print("=" * 65)

if __name__ == "__main__":
    main()
```

OUTPUT

Step 1: System Initialization & Menu Display

Menu Mapping: The system loads the menu dataset containing food and drink items, codes, categories, and prices.

User Interaction: It prompts the user to enter customer details: Student Name, Menu Item Code, Quantity, and Membership Status.

```
=================================================================
          PMJ POLYCAFE SALES ANALYSIS SYSTEM
=================================================================

--- PMJ POLYCAFE MENU ---
Code   | Item Name        | Category   | Price (RM)
-------------------------------------------------
1      | Nasi Lemak       | Food       | RM 4.00
2      | Chicken Rice     | Food       | RM 6.50
3      | Mee Goreng       | Food       | RM 5.00
4      | Iced Milo        | Drink      | RM 3.00
5      | Mineral Water    | Drink      | RM 1.50
-------------------------------------------------
```

Step 2: Matrix Processing using NumPy (Task 1)

Array Creation: Using np.full(quantity, price_per_item), the system creates a 1D NumPy matrix containing the price repeated across the item quantity.

Matrix Calculations:

- Accesses element 0 (array[0]) to verify unit price.
- Sums all array elements using .sum() to determine the initial transaction total.
- Evaluates array dimensions (.ndim) and shape (.shape).

```
======================================================================
TASK 1: MATRIX PROCESSING USING NUMPY
======================================================================
Enter Student Name: Adam
Enter Menu Item Code (1-5): 1
Enter Number of items purchased: 2
Membership Status (Yes/No): yes

--- Task 1 Output ---
Student Name             : Adam
Selected Item            : Nasi Lemak (Food)
Price per Item           : RM 4.00
Quantity Purchased       : 2
Generated NumPy Array    : [4. 4.]
First Element (array[0]): RM 4.00
Total Purchase (.sum()) : RM 8.00
Array Dimensions (.ndim): 1
Array Shape (.shape)     : (2,)
```

Step 3: DataFrame Structuring using Pandas (Task 2)

- Data Integration: The user's input from Task 1 (Nasi Lemak purchase) is combined with three existing student purchase records into a 2D Pandas DataFrame.
- Gross Purchase Calculation: Calculates Total Purchase for each record using:

```
======================================================================
TASK 2: DATA ORGANIZATION USING PANDAS
======================================================================

--- Complete Initial DataFrame ---
   Student Name          Item Category  Quantity Membership  Price per Item  Total Purchase
0          Adam     Nasi Lemak     Food         2        Yes             4.0             8.0
1        Irsyad     Nasi Lemak     Food         2         No             4.0             8.0
2   Ambok irfan      Iced Milo    Drink         4        Yes             3.0            12.0
3  Fakhrul iman  Mineral Water    Drink         5         No             1.5             7.5
```

Step 4: Discount & Payment Engine (Task 3)

- Conditional Business Logic: The system checks the Membership column for each row:
  If Membership = "Yes": Grants a 10% discount ($\text{Total Purchase} \times 0.10$).
  If Membership = "No": Sets Discount to RM 0.00.

```
============================================================
TASK 3: MEMBERSHIP DISCOUNT AND FINAL PAYMENT
============================================================

--- Updated DataFrame with Discount and Final Payment ---
   Student Name          Item Category  Quantity Membership  Price per Item  Total Purchase  Discount  Final Payment
0          Adam     Nasi Lemak     Food         2        Yes             4.0             8.0       0.8            7.2
1        Irsyad     Nasi Lemak     Food         2         No             4.0             8.0       0.0            8.0
2   Ambok irfan      Iced Milo    Drink         4        Yes             3.0            12.0       1.2           10.8
3  Fakhrul iman  Mineral Water    Drink         5         No             1.5             7.5       0.0            7.5
```

Step 5: Data Cleaning & Manipulation Pipeline (Task 4)

- **Duplicate Handling**:

  Appends a temporary duplicate row using pd.concat().

  Detects duplicate entries using .duplicated().

  Purges duplicate entries using .drop_duplicates() to restore data integrity.

- **Missing Value Imputation**:

  Simulates a missing entry by inserting None into row index 2.

  Identifies missing values using .isnull().

  Fills missing values with "No" using .fillna('No').

```
============================================================
TASK 4: DATA MANIPULATION
============================================================

DataFrame after adding 1 duplicate row:
   Student Name          Item Category  Quantity Membership  Price per Item  Total Purchase  Discount  Final Payment
0          Adam     Nasi Lemak     Food         2        Yes             4.0             8.0       0.8            7.2
1        Irsyad     Nasi Lemak     Food         2         No             4.0             8.0       0.0            8.0
2   Ambok irfan      Iced Milo    Drink         4        Yes             3.0            12.0       1.2           10.8
3  Fakhrul iman  Mineral Water    Drink         5         No             1.5             7.5       0.0            7.5
4        Irsyad     Nasi Lemak     Food         2         No             4.0             8.0       0.0            8.0
```

```
Detect Duplicate Rows using .duplicated():
0    False
1    False
2    False
3    False
4     True
dtype: bool
```

DataFrame after removing duplicate rows using .drop_duplicates():

	Student Name	Item	Category	Quantity	Membership	Price per Item	Total Purchase	Discount	Final Payment
0	Adam	Nasi Lemak	Food	2	Yes	4.0	8.0	0.8	7.2
1	Irsyad	Nasi Lemak	Food	2	No	4.0	8.0	0.0	8.0
2	Ambok irfan	Iced Milo	Drink	4	Yes	3.0	12.0	1.2	10.8
3	Fakhrul iman	Mineral Water	Drink	5	No	1.5	7.5	0.0	7.5

DataFrame after inserting ONE missing value (None) at Index 2:

	Student Name	Item	Category	Quantity	Membership	Price per Item	Total Purchase	Discount	Final Payment
0	Adam	Nasi Lemak	Food	2	Yes	4.0	8.0	0.8	7.2
1	Irsyad	Nasi Lemak	Food	2	No	4.0	8.0	0.0	8.0
2	Ambok irfan	Iced Milo	Drink	4	NaN	3.0	12.0	1.2	10.8
3	Fakhrul iman	Mineral Water	Drink	5	No	1.5	7.5	0.0	7.5

Detect Missing Values using .isnull():

	Student Name	Item	Category	Quantity	Membership	Price per Item	Total Purchase	Discount	Final Payment
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False

DataFrame after replacing missing Membership with 'No' using .fillna():

	Student Name	Item	Category	Quantity	Membership	Price per Item	Total Purchase	Discount	Final Payment
0	Adam	Nasi Lemak	Food	2	Yes	4.0	8.0	0.8	7.2
1	Irsyad	Nasi Lemak	Food	2	No	4.0	8.0	0.0	8.0
2	Ambok irfan	Iced Milo	Drink	4	No	3.0	12.0	1.2	10.8
3	Fakhrul iman	Mineral Water	Drink	5	No	1.5	7.5	0.0	7.5

Final Cleaned DataFrame ---

	Student Name	Item	Category	Quantity	Membership	Price per Item	Total Purchase	Discount	Final Payment
0	Adam	Nasi Lemak	Food	2	Yes	4.0	8.0	0.8	7.2
1	Irsyad	Nasi Lemak	Food	2	No	4.0	8.0	0.0	8.0
2	Ambok irfan	Iced Milo	Drink	4	No	3.0	12.0	0.0	12.0
3	Fakhrul iman	Mineral Water	Drink	5	No	1.5	7.5	0.0	7.5

Step 6: Data Aggregation & Analysis

- Category: The unique product categories found in the menu (Drink and Food).
- Total_Purchase: The total gross sales amount (before discount) for all items sold within that category, calculated using .sum().
- Total_Final_Payment: The total net revenue collected (after applying membership discounts) within that category, calculated using .sum().
- Transaction_Count: The total number of individual purchases/orders recorded for that category, calculated using .count().

```
======================================================================
TASK 5: DATA ANALYSIS (GROUPING)
======================================================================

--- Sales Analysis by Category ---
Category  Total_Purchase  Total_Final_Payment  Transaction_Count
   Drink            19.5                 19.5                  2
    Food            16.0                 15.2                  2

======================================================================
       SYSTEM PROCESSING COMPLETED SUCCESSFULLY
======================================================================
```

	NAME	MOHAMMAD IRSYAD HAKIMI BIN SABARUDIN	**MARKS /100**
	CLASS	DIT5F	
	REGISTRATION No.	25DIT24F1089	
	CODE/COURSE NAME	DFK50083 PYTHON PROGRAMMING	
	ASSESSMENT	PRACTICAL WORK 4	
	LECTURER	TS. HANIS BASIRA BINTI ABU HASAN	

Display suitable techniques in mathematical computation and software application using python programming. (P3, PLO3)

TOPIC: 4.1 Explain the Introduction to Matrix Processing

4.2 Explain Analysis and manipulation tools

Criteria	Excellent 4	Good 3	Moderate 2	Poor 1	Weightage	Score	Mark
Task 1: NumPy Matrix Processing	NumPy array correctly created using np.full(). Menu data, array, indexing, sum, ndim and shape are displayed accurately.	NumPy array is correct with minor output or logic issues.	NumPy array is created but some required outputs are incorrect or missing.	NumPy implementation is incomplete or incorrect.	/30	___ /4 * 30	
Task 2: DataFrame Creation and Organization	DataFrame is created correctly with all required columns and at least four valid records. Total Purchase is calculated correctly.	DataFrame is mostly correct with minor structure or calculation errors.	DataFrame is created but some columns, records or values are missing/incorrect.	DataFrame is incomplete or incorrect.	/20	___ /4 * 20	
Task 3: Membership Discount and Final Payment	Discount and Final Payment are implemented correctly for members and non-members with accurate calculations.	Minor errors occur in the membership condition or calculation.	Logic is partially correct but some results are incorrect.	Columns are missing or incorrectly implemented.	/15	___ /4 * 15	

Task 4: Data Manipulation (Duplicate & Missing Values)	Duplicate and missing values are correctly detected and handled using duplicated(), drop_duplicates(), isnull() and fillna().	Minor errors occur in detection or handling.	Partial implementation; some steps are missing or incorrect.	Data manipulation is not implemented correctly.	/15	___ /4 * 15	
Criteria	**Excellent 4**	**Good 3**	**Moderate 2**	**Poor 1**	**Weightage**	**Score**	**Mark**
Task 5: Data Analysis (Grouping)	groupby() is correctly implemented. Total Purchase, Final Payment and transaction count are accurate and meaningful.	Minor errors occur in grouping or aggregation.	Grouping is attempted but some results are incorrect.	groupby () is not implemented or is incorrect.	/10	___ /4 * 10	
Code Readability and Documentation	Code is well-structured with clear variable names, appropriate comments and easy-to-follow organization.	Code is mostly clear with minor readability issues.	Some structure is shown but comments or naming are limited.	Code structure is poor and difficult to understand.	/5	___ /4 * 5	
Program Execution and Output	Program runs smoothly without errors. Output is complete, clear and accurate.	Minor runtime or output issues are present.	Program runs with several errors affecting output.	Program does not run properly.	/5	___ /4 * 5	
					TOTAL	**100**	

NOTE: Zero (0) may be awarded if the student does not answer/perform the required criterion.